

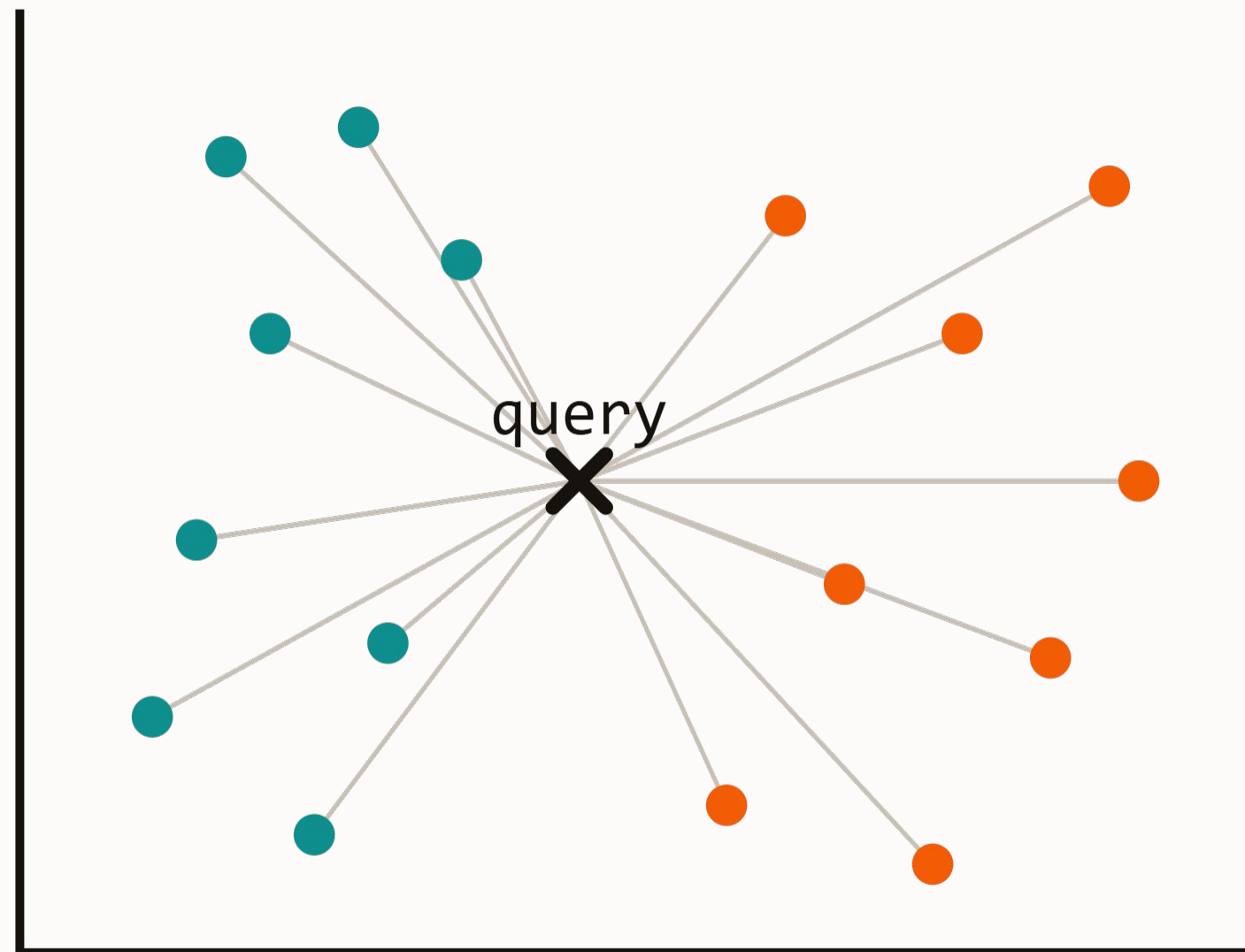
VARIANTS OF KNN

Optimising KNN: KD Tree and Ball Tree

Arrange the training points into a tree once, and every prediction after that can ignore most of them.

WHAT WE ARE FIXING

Brute Search Measures Everything



Measure to point 1, point 2, point 3, all the way to the last one. Then sort and keep the closest few.

$O(n)$ the work grows with the number of rows

Fine for a thousand rows. Painful for ten million, and it happens on every single prediction.

The waste is obvious in the picture. Most of those lines go to points that were never going to be close.

■ THE SHARED IDEA

Split the points into halves once, then throw away a half at every step.

Build once

At fit time, arrange the training points into a binary tree. This costs something, and you pay it once.

Use every time

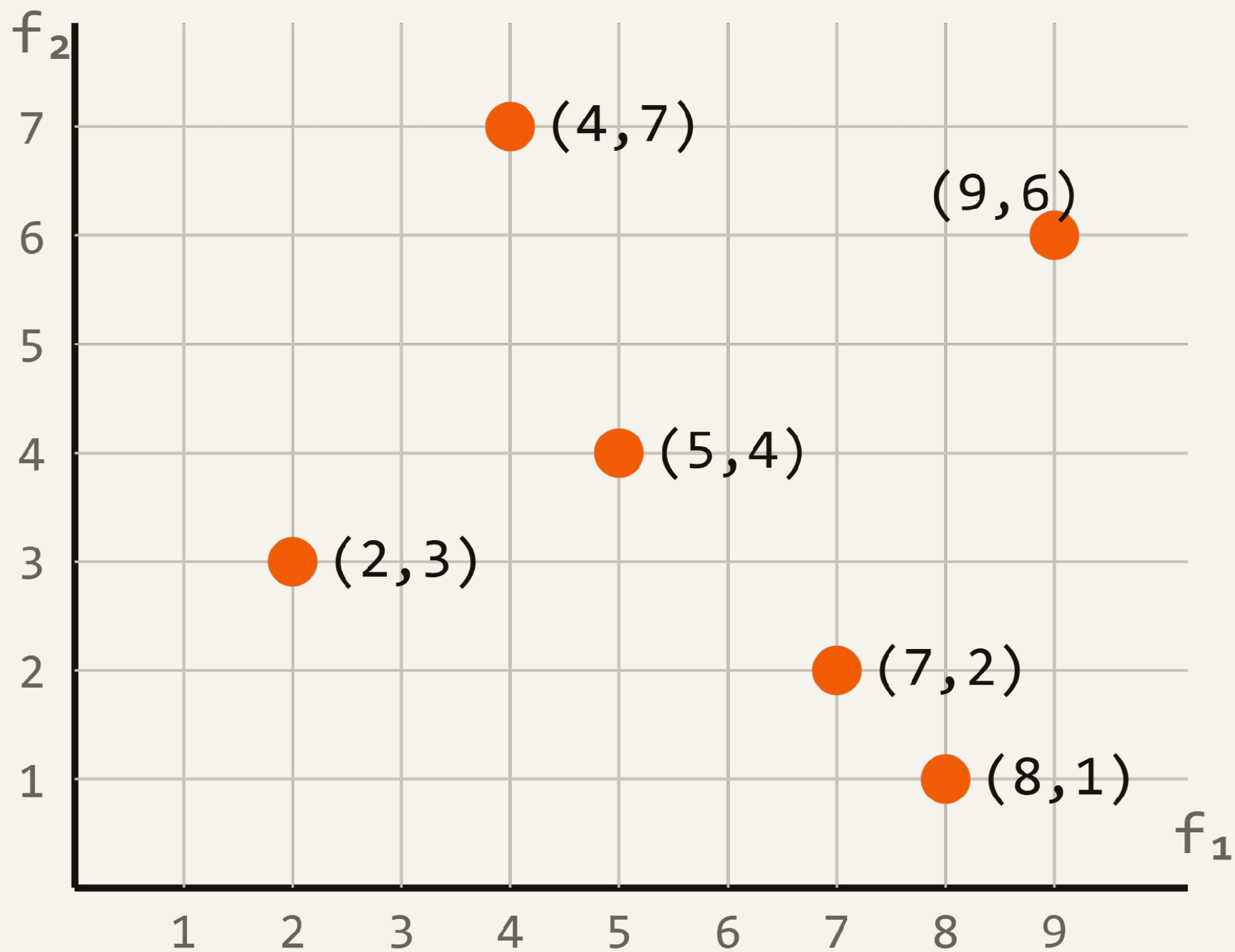
Each query walks down the tree. At every level one entire branch is ruled out without measuring anything inside it.

Two ways to do it

KD tree splits with straight lines along one feature at a time. Ball tree groups points into spheres.

Six Points, Two Features

f_1	f_2
7	2
5	4
9	6
2	3
4	7
8	1



What One Split Actually Does

01

Pick a feature

At the top use f_1 . One level down use f_2 . Then back to f_1 , and so on.

02

Find its median

Sort that feature's values and take the middle one, so each side gets half the points.

03

Round to a real point

If the median is 6.5 and no point sits at 6.5, use 5 or 7. We always split at a point that exists.

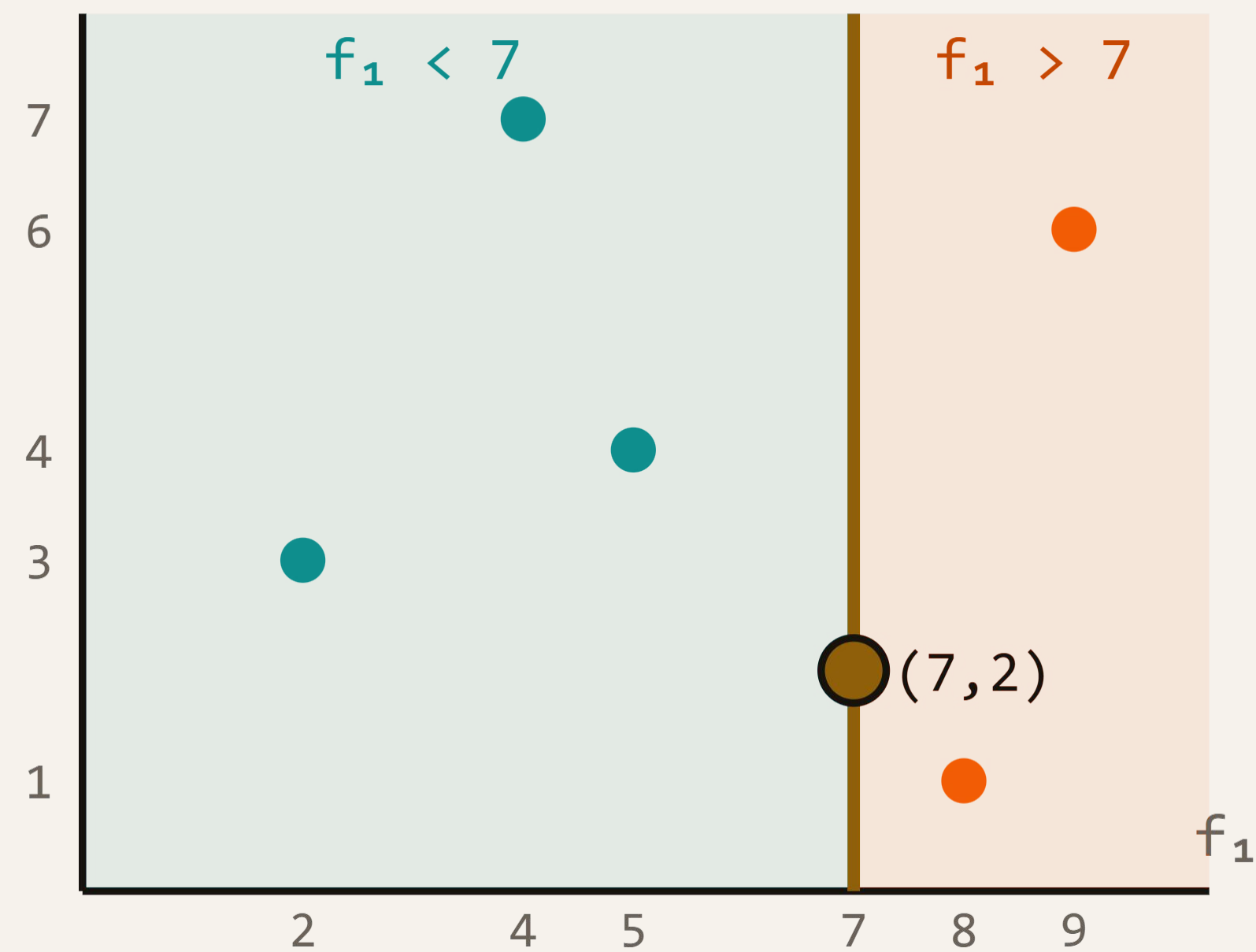
04

That point becomes a node

The leftover points split in two: below it go left, above it go right. Then repeat inside each side.

Four steps, repeated until every point has a place. The next slides do exactly this, one round at a time.

The First Split Makes the Root



F_1 SORTED

2 · 4 · 5 · 7 · 8 · 9 → 6.5 → 7

Root of the tree: (7,2)

LEFT · $f_1 < 7$

(5,4) (2,3) (4,7)

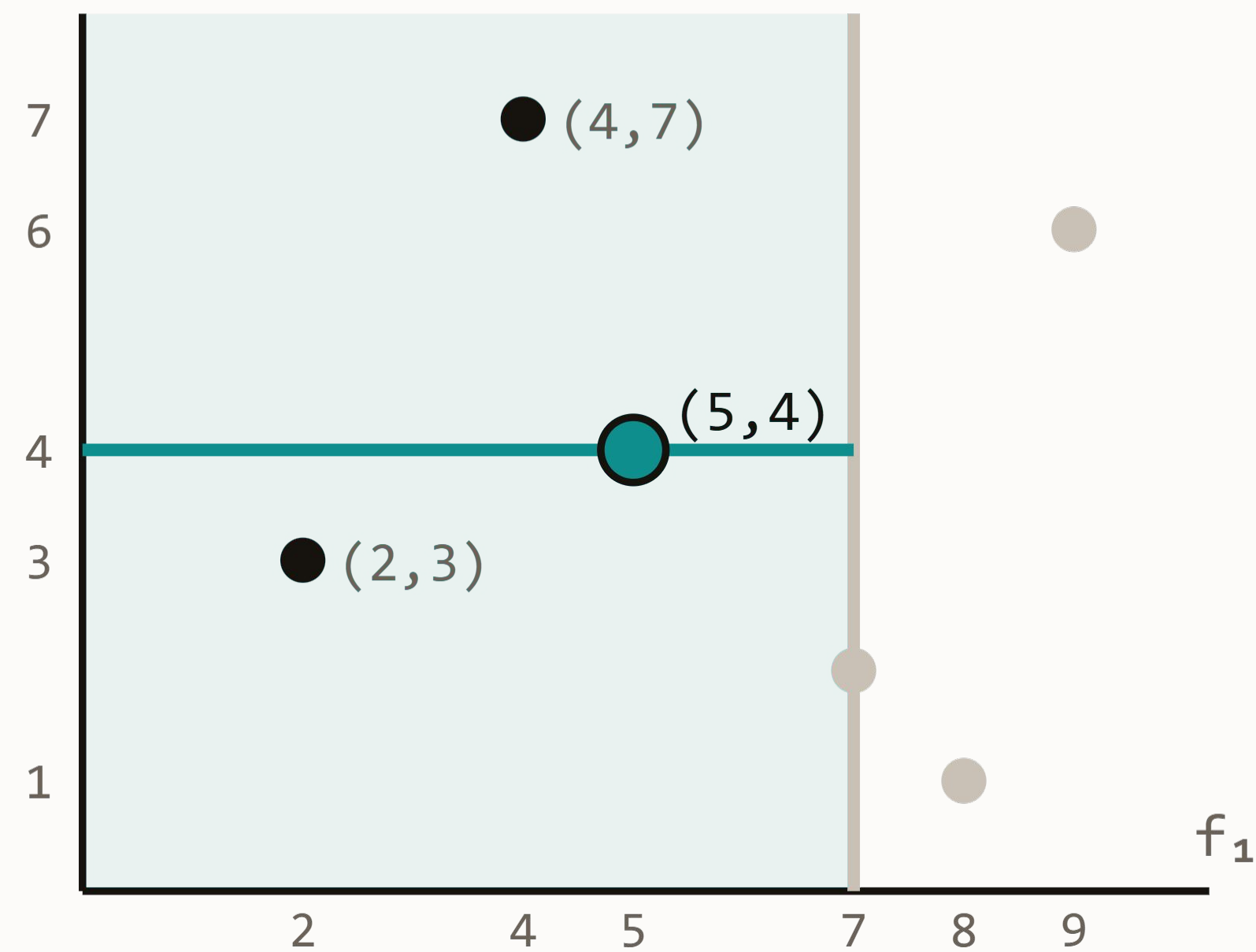
RIGHT · $f_1 > 7$

(9,6) (8,1)

Those two groups are now the left and right children. Each gets split again in round two.



Split the Left Group



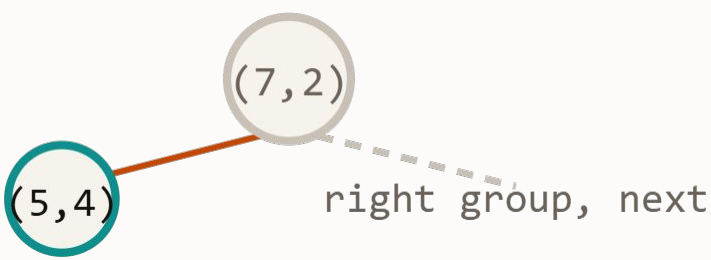
THE 3 POINTS IN THIS GROUP
 $(5,4)$ $(2,3)$ $(4,7)$

THEIR F_2 VALUES, SORTED
 $3 \cdot 4 \cdot 7 \rightarrow$ median is 4

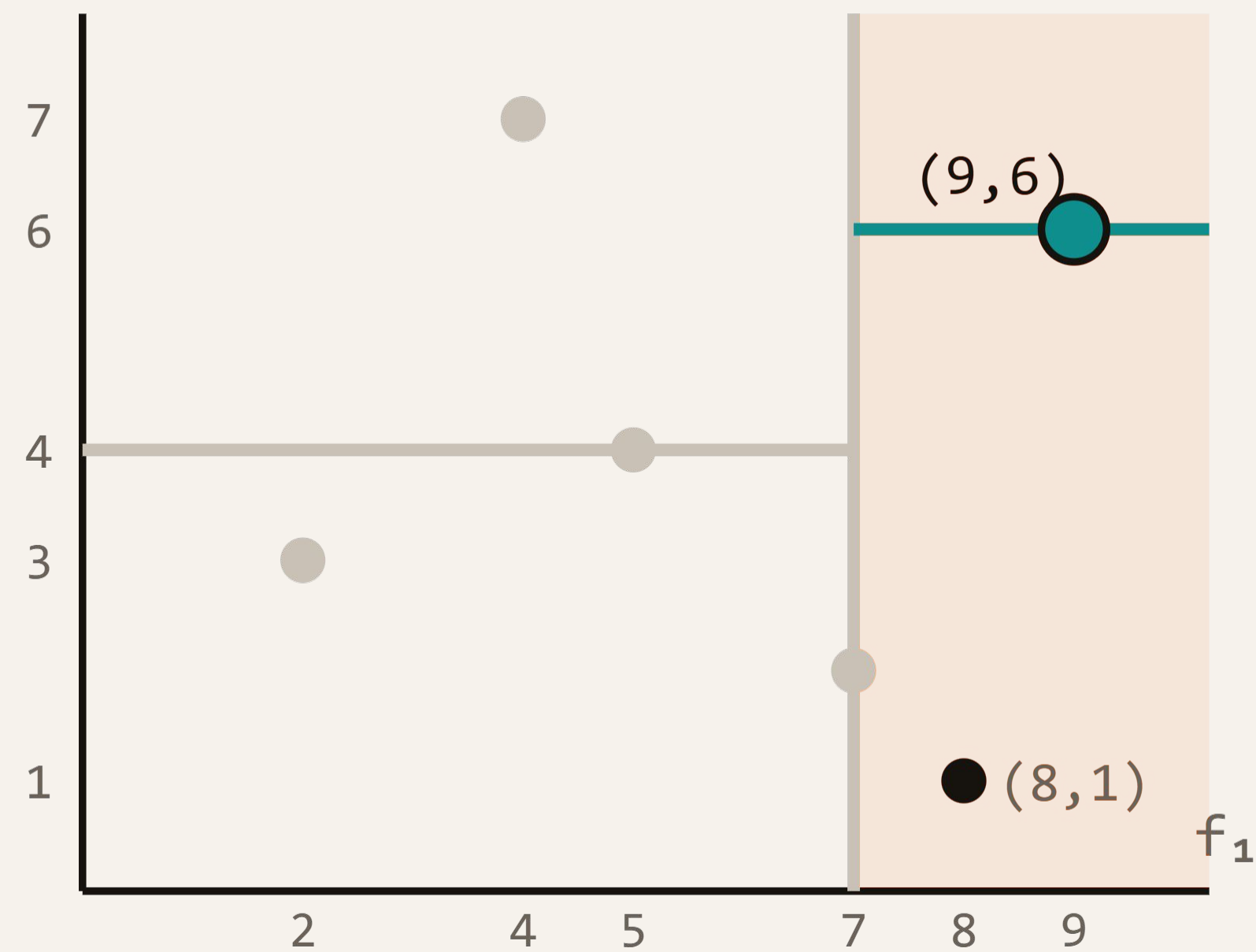
Left child of the root: $(5,4)$

The line stops at the vertical one. This split belongs to the left group only, so it does not cross into the right side.

Two points remain here: $(2,3)$ below the line, $(4,7)$ above it.



Split the Right Group the Same Way



THE 2 POINTS IN THIS GROUP

$(9,6)$ $(8,1)$

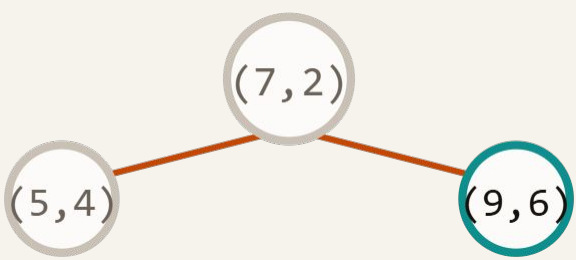
THEIR F_2 VALUES, SORTED

$1 \cdot 6 \rightarrow 3.5 \rightarrow \text{round to } 6$

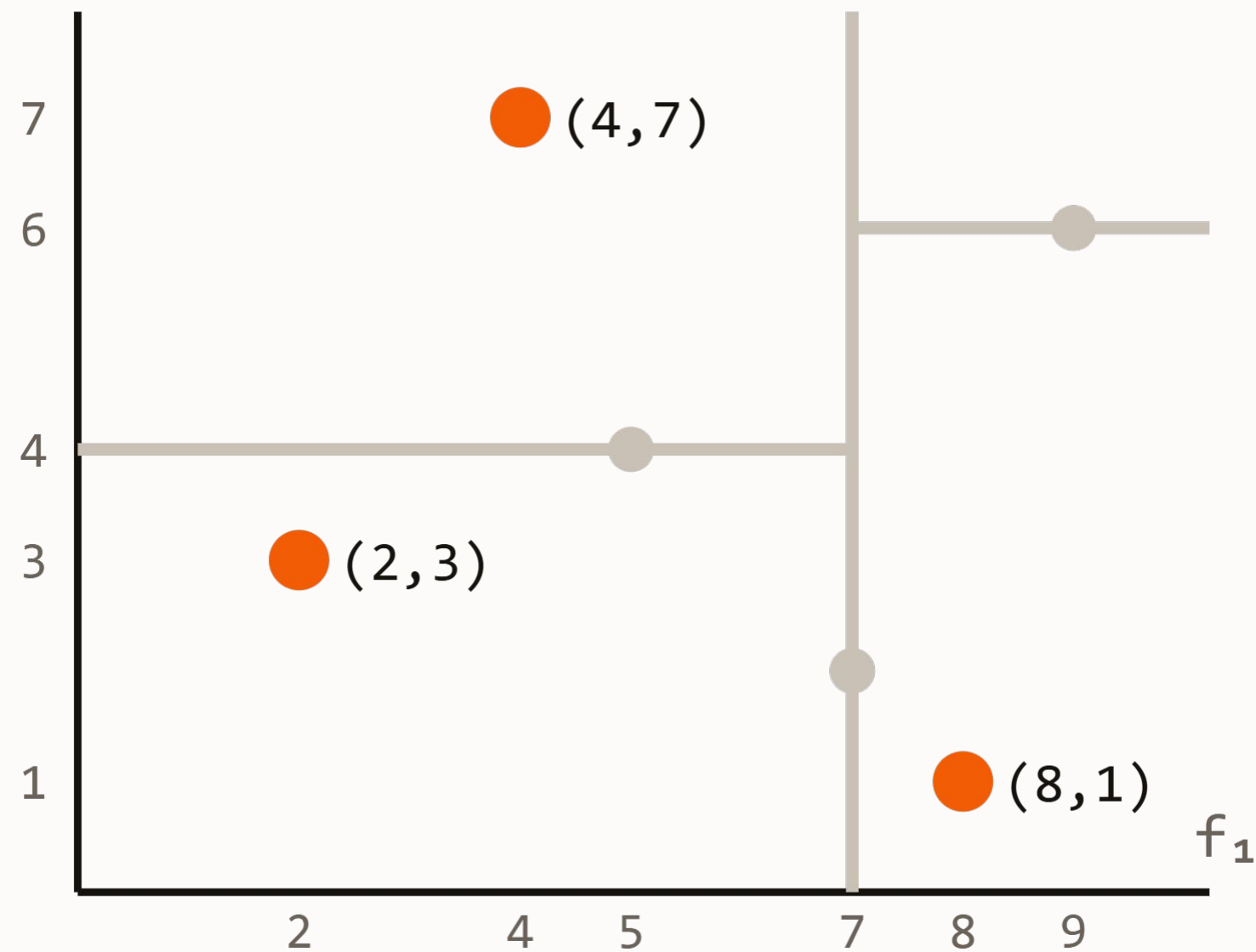
Right child of the root: $(9,6)$

Exactly the same four steps as the left group. Nothing about the procedure changed.

One point remains here: $(8,1)$, below the line.



The Last Three Points Become Leaves



$(2, 3)$

Left group, split at $f_2 = 4$. Its f_2 is 3, below 4, so it goes left of $(5, 4)$.

$(4, 7)$

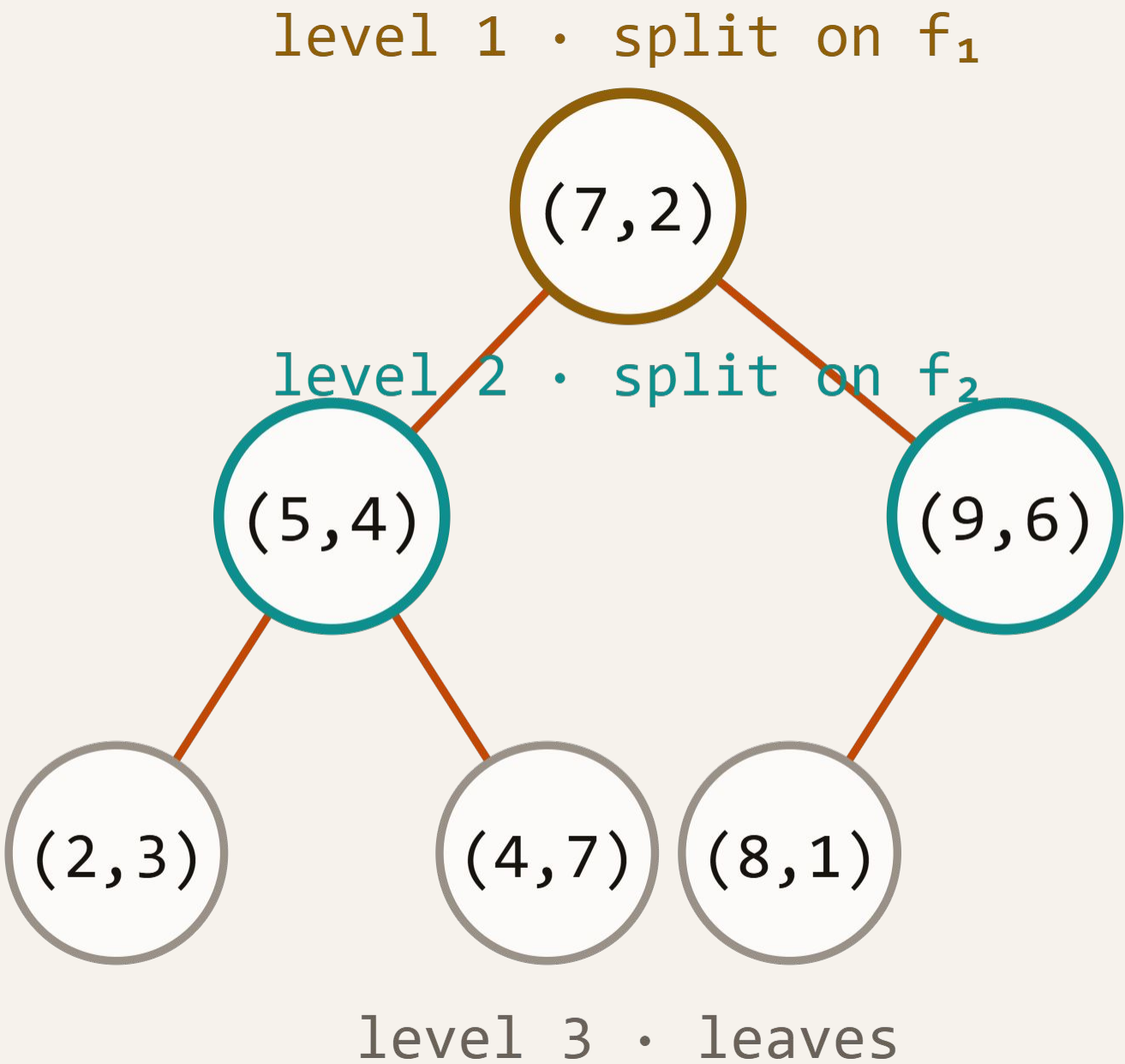
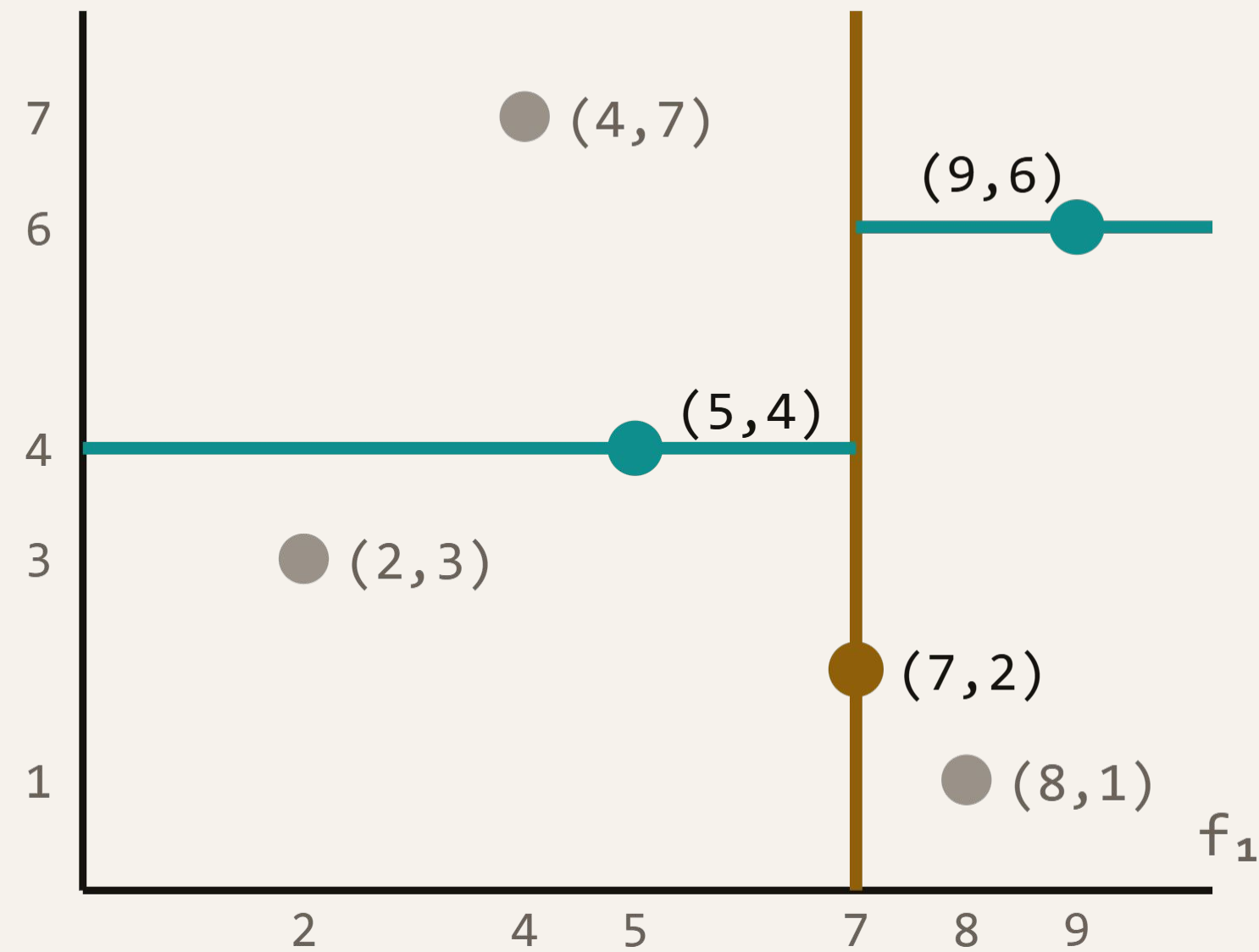
Same group. Its f_2 is 7, above 4, so it goes right of $(5, 4)$.

$(8, 1)$

Right group, split at $f_2 = 6$. Its f_2 is 1, below 6, so it goes left of $(9, 6)$.

Each group now holds a single point, so there is nothing left to split. These three are the leaves, and the tree is finished.

The Splits Are the Tree



Every region on the left matches exactly one path down the tree on the right. They are the same information, drawn two ways.

When Is It Built, and Which Point Starts It?

IS IT BUILT BEFOREHAND?

Yes. The tree is built once, when you call `fit`, before any prediction is made.

This is the one piece of real work KNN does at training time. Every query afterwards reuses the same tree.

WHICH POINT IS THE ROOT?

Not one you pick. Sort the first feature, take the median, and whichever point sits there becomes the root.

COULD YOU START ANYWHERE?

You could, and the tree would still be correct. It would just stop being fast, which was the entire point.

`median root`

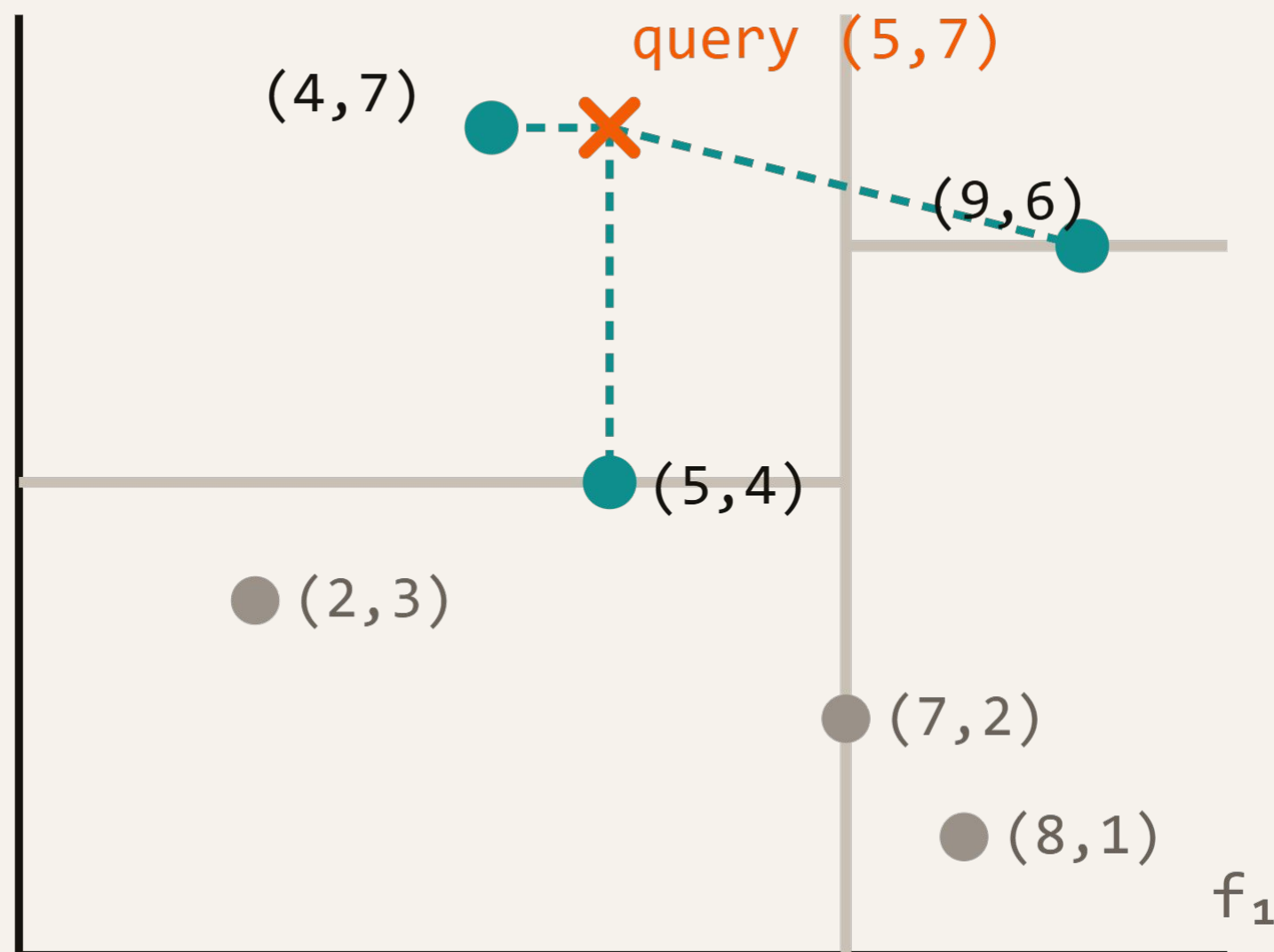
3 points each side, then 1 and 1. A short, balanced tree.

`extreme root, say $f_1 = 2$`

0 points on one side and 5 on the other, then 0 and 4. The tree stretches into a chain.

Walking down a chain means visiting every point anyway, so you are back to brute search with extra steps. The median is what keeps it short.

Walk Down, Then Backtrack



01 At $(7,2)$: f_1 is 5, below 7, so go left. The whole right branch is set aside.

02 At $(5,4)$: f_2 is 7, above 4, so go right and land on $(4,7)$.

NEAREST, IN ORDER

1st $(4,7)$ · $d = 1.00$

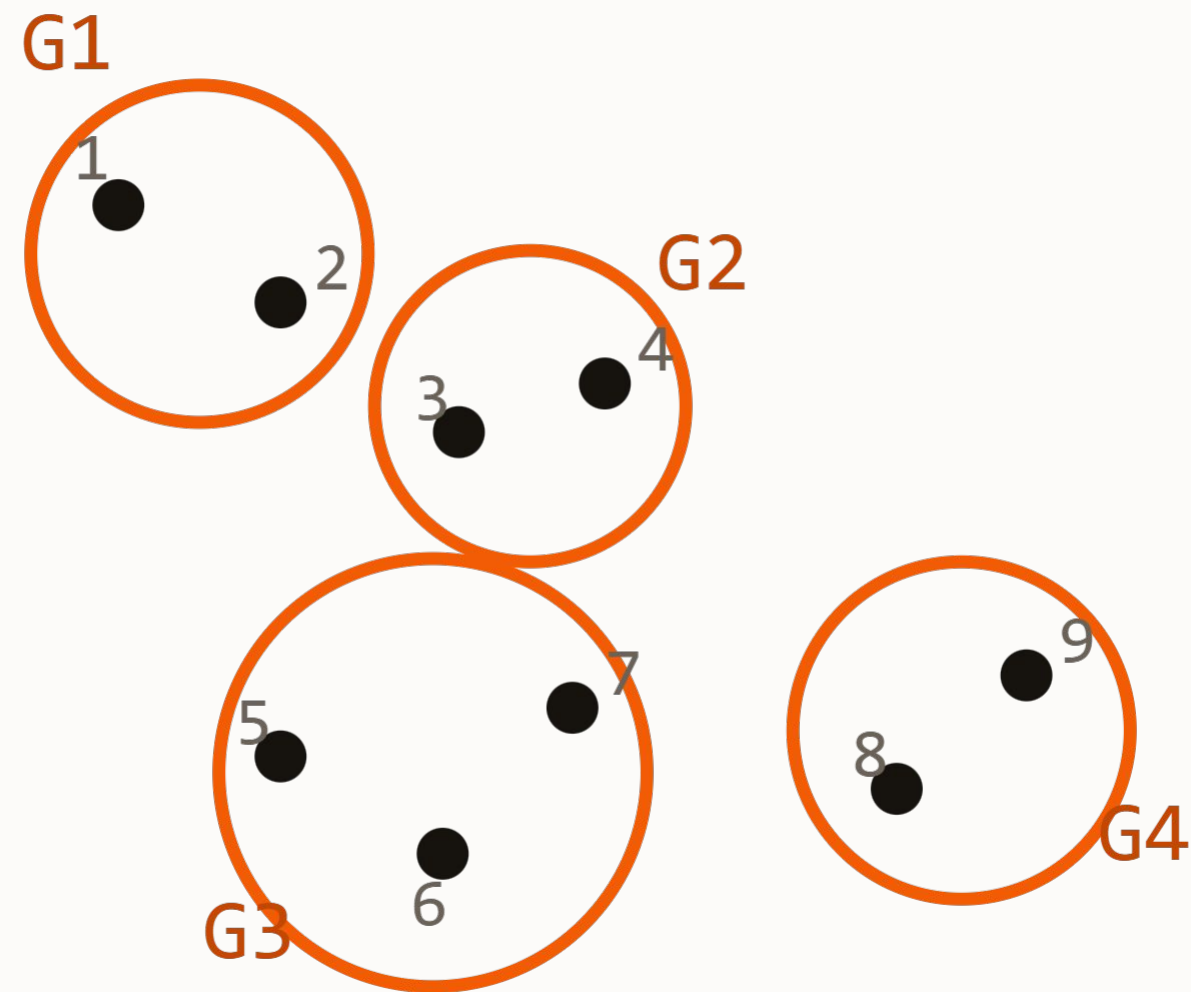
2nd $(5,4)$ · $d = 3.00$

3rd $(9,6)$ · $d = 4.12$

The third nearest sits in the branch we set aside at the very first step. So the skipped branch has to be re-checked on the way back up.

That return journey is called backtracking, and it is the weak spot of the KD tree.

Wrap Nearby Points in Spheres



No straight lines this time. Nearby points are collected into spheres, which is where the name ball tree comes from.

G1 → 1, 2

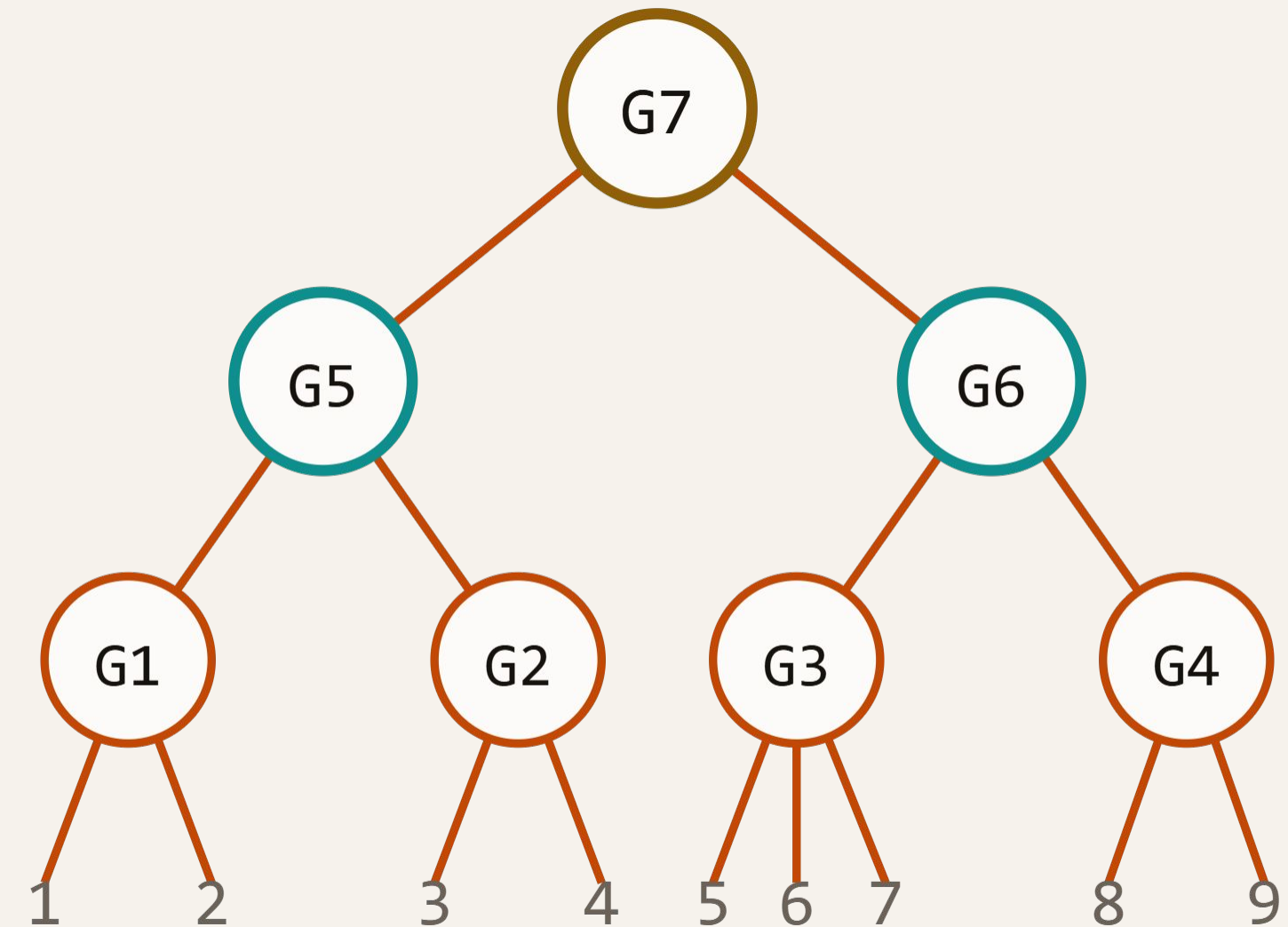
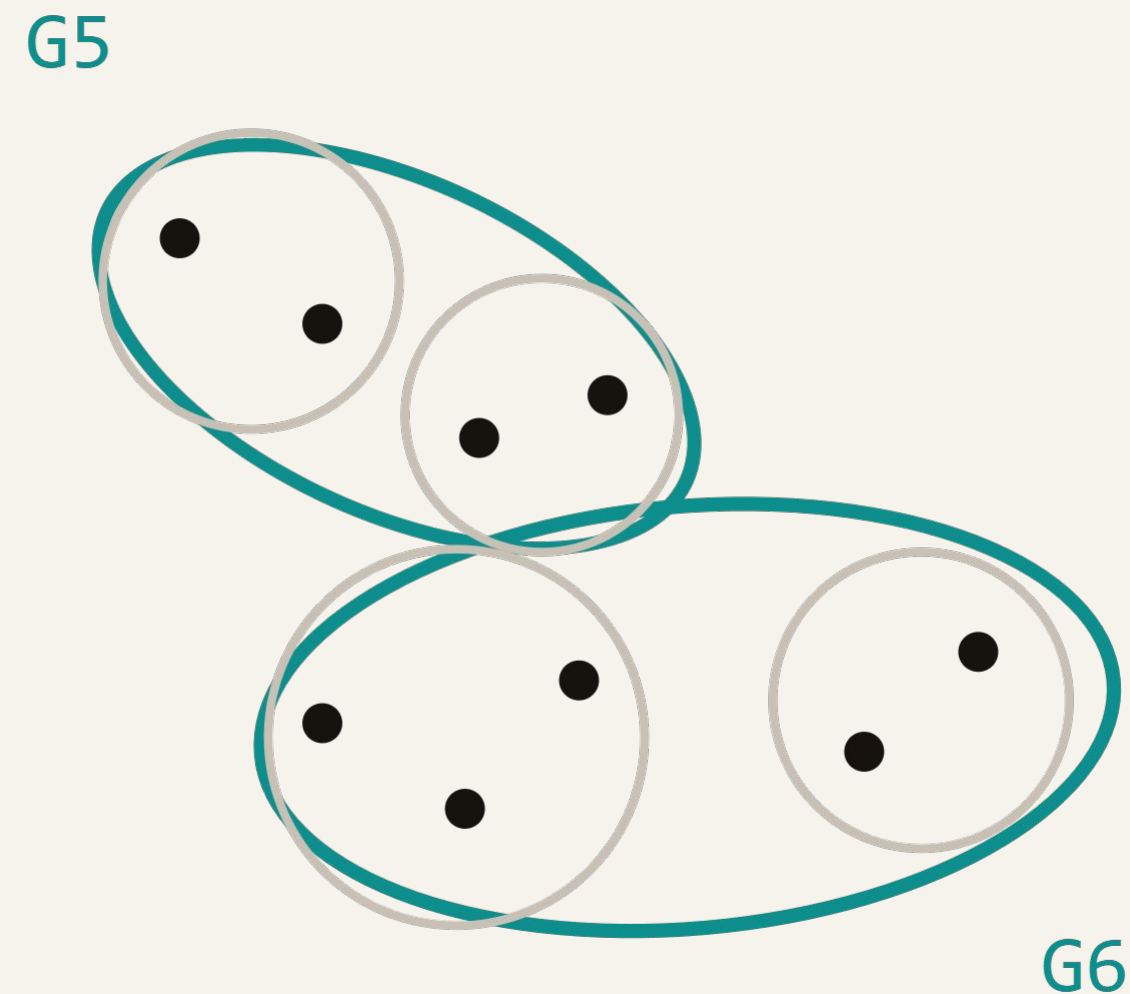
G2 → 3, 4

G3 → 5, 6, 7

G4 → 8, 9

Every point belongs to exactly one group. These four are the bottom level of the tree.

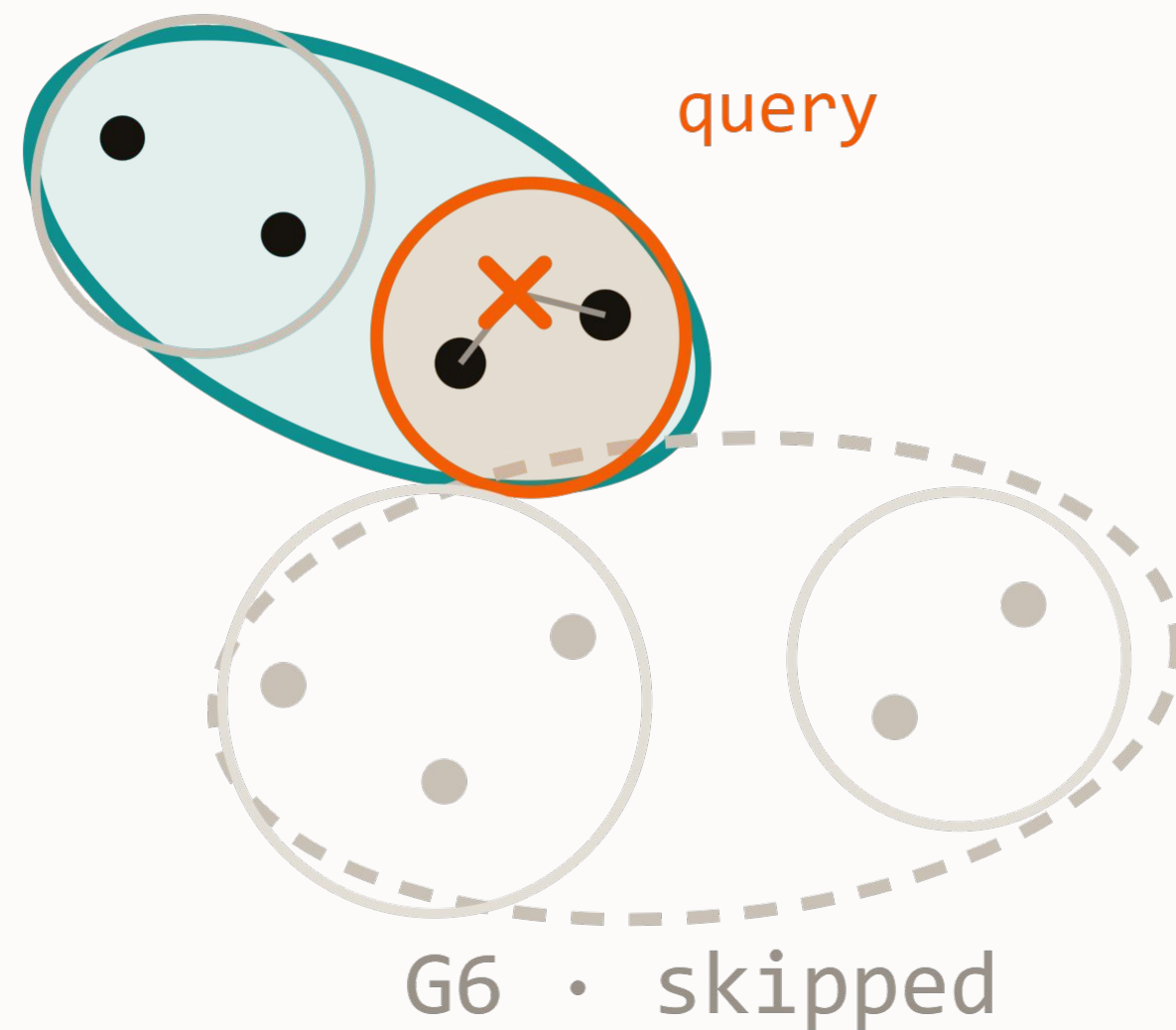
Merge the Nearest Groups, Then Merge Again



G1 and G2 merge into G5, G3 and G4 merge into G6, and those two merge into G7, the root. Read from the top, it is a binary tree, built from spheres instead of lines.

Pick a Sphere, Discard the Rest

G5



01 At the root, which child sphere is the query closer to? It falls in G5.

02 Every point under G6 is out. Five of the nine, and not one of them was measured.

03 Ask again inside G5: G1 or G2. Land in G2, and measure only the points there.

A sphere either covers the area around the query or it does not, so far less backtracking is needed than with a KD tree.

Once You Reach One Group, Is It Just Normal KNN?

YES, EXACTLY THAT

Inside the final group you measure the distance to every point in it and keep the closest ones. Same formula, same comparison.

The tree never replaced the distance calculation. It only changed how many points you run it on.

without a tree

measure against all n points

with a tree

measure against one group

WHAT IF THE GROUP IS TOO SMALL?

If the group holds fewer points than your K , you cannot stop there. You walk back up and open the neighbouring group until you have collected K .

A group of 2 cannot answer a request for 5 neighbours on its own.

In scikit-learn the size of these bottom groups is `leaf_size`, which defaults to 30, so a leaf normally holds more than enough points for an ordinary K .

SIDE BY SIDE

KD Tree and Ball Tree Compared

KD tree

Splits with straight lines, one feature at a time

Faster to build

Strong while the feature count stays small, roughly under 20

Needs backtracking on every query

Ball tree

Groups points into spheres, no axis involved

Slower to build

Holds up far better with many features

Much less backtracking

Both give exactly the same neighbours as brute search. Only the speed of finding them differs.

As features pile up, box-shaped regions stop separating points usefully. Spheres keep working.

Which One Do You Pick? Usually Neither

Scikit-learn's `algorithm` setting defaults to `auto` . It inspects your data and chooses for you.

`brute` very small dataset, where building a tree costs more than it saves

`kd_tree` a small number of features

`ball_tree` a larger number of features

You can override it by name if you want to compare timings yourself.

THE WARNING THAT MATTERS

Neither tree saves you when the number of features is genuinely large.

Past roughly thirty features, every point ends up about the same distance from every other point, so "nearest" stops meaning much. The tree cannot skip branches when nothing is clearly closer.

This is the curse of dimensionality, and it limits KNN itself, not just the tree.

The fix is to cut the number of features down first, with something like PCA, rather than switching trees.

Build Once, Skip Branches Forever

01 Plain KNN measures every point, every prediction. That is $O(n)$.

02 Build a binary tree once, then discard whole branches per query.

03 KD tree: straight lines on alternating features, plus backtracking.

04 Ball tree: spheres, slower to build, far better with many features.

Neither changes which neighbours you get. Only the time taken to find them.

Scikit-learn's auto setting picks for you, and it is usually the right call.

The real limit is the number of features. Reduce those before blaming the tree.